



Essay / Assignment Title: From raw data to meaningful insights with machine learning

Programme title: BSc (Hons) Computer Science and Digitisation.

Name: SIDDHANT SAINATH DANE

Year: 2026

CONTENTS



1. Introduction
2. Task 1 – Linear Regression Analysis: Store 1
3. Task 2 – Polynomial Regression Analysis: Store 2
4. Task 3 – SVM Binary Classification with 3D Visualization
5. Concluding Remarks
6. Bibliography
- 7. Appendix (GitHub Repository Link)**

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters): SIDDHANT DANE

.....

Date: 10/06/2026

INTRODUCTION

The utility of the data lies in its ability to generate the knowledge. In the business settings, numbers locked away in the spreadsheets do not provide much insight on their own, but the application of the various analysis models turns the numbers into the action points. This report explores the three models relevant to the predictive analysis, such as the linear regression, the polynomial regression, and the Support Vector Machine (SVM) .

In the Task 1, we explore the possibility that increased advertisements on the weekends will result in the linearly predictable store sales throughout the week.

In the Task 2, we check this hypothesis against a more complicated dataset where there is no clear relationship between the advertisement and the sales, and a different approach must be adopted for selecting the right model.

The Task 3 shifts to the classification by building an SVM model with the RBF kernel based on the Breast Cancer Wisconsin dataset with the three predictor variables used to determine whether a tumour is malignant or a benign.

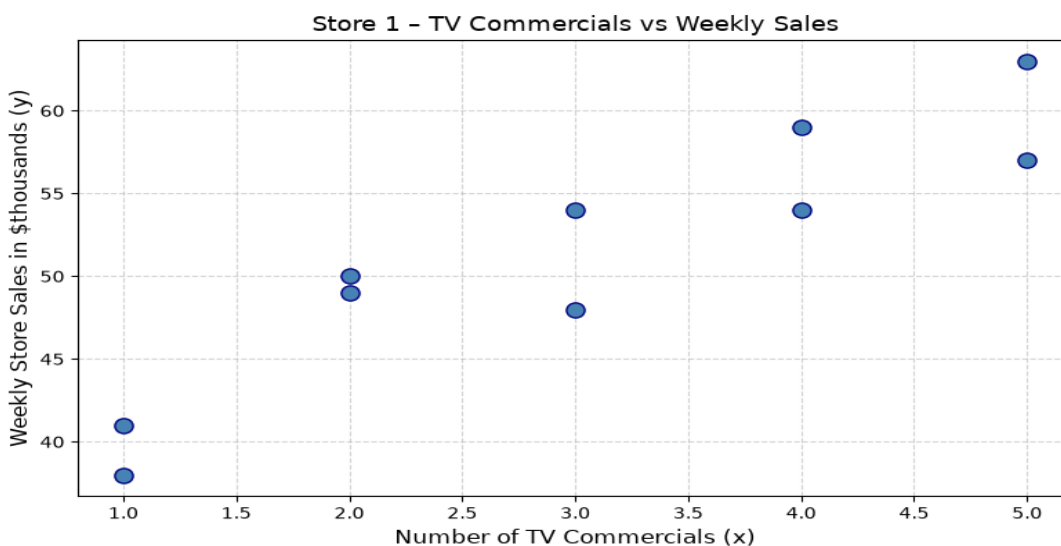
All the tasks follow the same framework for the performing a proper analysis, which consists of the understanding the data, the choosing a relevant model, applying it effectively, and evaluating it objectively. This step-by-step approach, rather than the difficulty of the algorithms used , determines the effectiveness of any analysis (James et al., 2021).

Task 1 – Linear Regression Analysis: Store 1

1.1 Data Visualization

For the Store 1, the number of the pairs of observation is the ten. In the particular, the $x=[2,5,1,3,4,1,5,3,4,2]$ refers to the number of the weekend television commercials while the $y=[50,57,41,54,54,38,63,48,59,49]$ shows the associated sales revenue per week in the thousands of dollars. The following script produces the scatter plot which is the very first step toward fitting a regression line as the inspection of the raw data will help to grasp its nature.

```
Predictive_Analysis.py X
Predictive_Analysis.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy import stats
4
5  # — Dataset —
6  x = np.array([2, 5, 1, 3, 4, 1, 5, 3, 4, 2])
7  y = np.array([50, 57, 41, 54, 54, 38, 63, 48, 59, 49])
8
9  # — Scatter plot —
10 plt.figure(figsize=(8, 5))
11 plt.scatter(x, y, color='steelblue', s=90, edgecolors='navy', zorder=5)
12 plt.xlabel("Number of TV Commercials (x)", fontsize=12)
13 plt.ylabel("Weekly Store Sales in $thousands (y)", fontsize=12)
14 plt.title("Store 1 | TV Commercials vs Weekly Sales", fontsize=13)
15 plt.grid(True, linestyle='--', alpha=0.55)
16 plt.tight_layout()
17 plt.show()
```



1.2 Justification for Linear Regression and Statistical Measure

The Linear regression assumes a straight-line relationship of the form:

$$\hat{y} = \beta_0 + \beta_1 x$$

In the Linear regression, the β_0 and the β_1 stand for the intercept and the slope, respectively. The model must satisfy the following conditions to be considered as adequate: The Relation between the x and the y has to be nearly Linear, The Residuals have to be normally distributed with the constant variance (homoscedasticity) and independence. The latter condition is met in the current case.

The appropriate way to test the strength of linear dependency is via the Pearson correlation coefficient (r) defined as follows:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

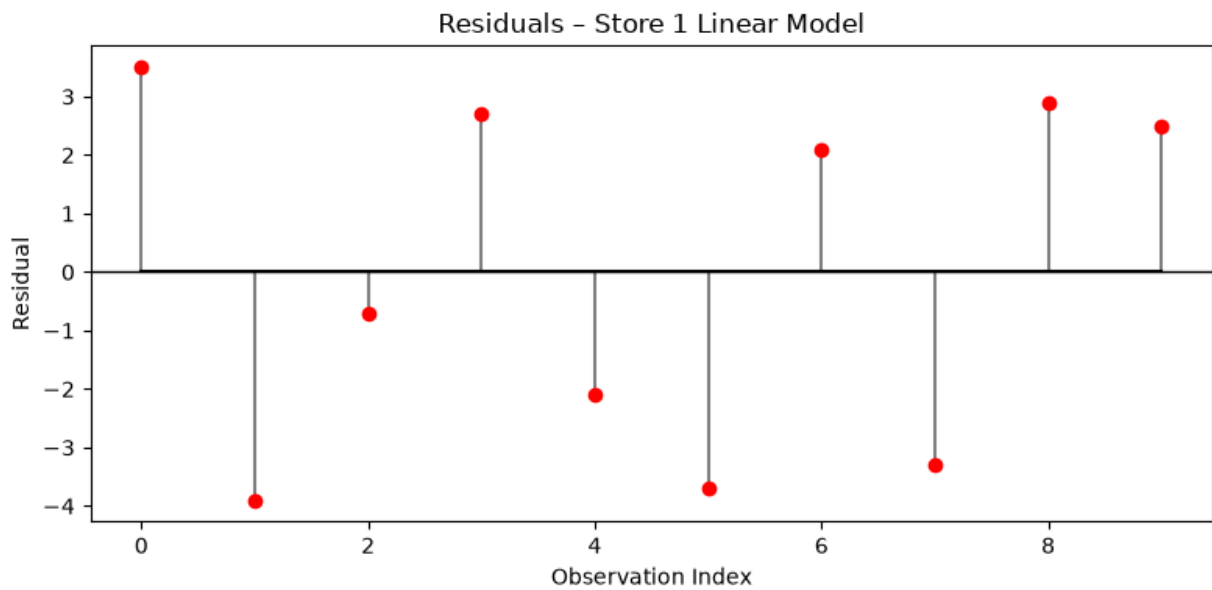
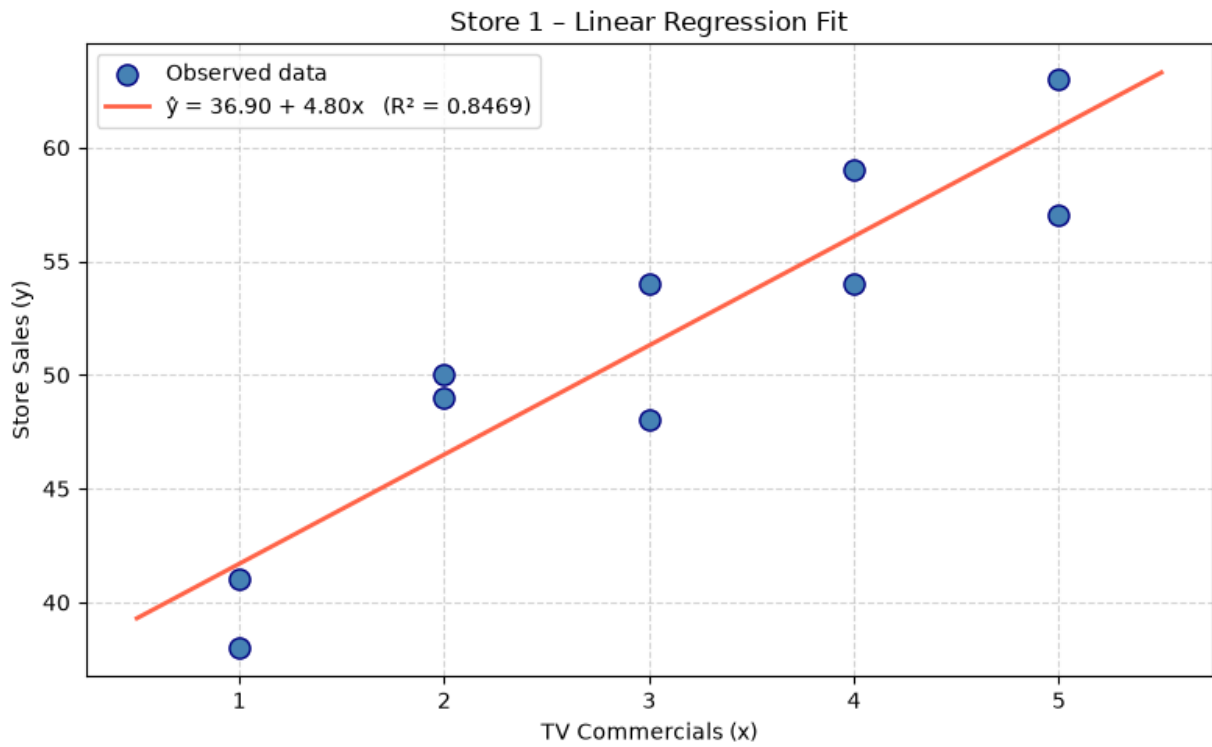
For this dataset: $\bar{x}=3.00$, $\bar{y}=51.30$. Computing the formula across all ten observations yields:

$r=0.9203$, $p\text{-value}=0.000160$

The obtained value of the r (0.92) demonstrates the high correlation between the variables under the study. The significance level ($p=0.00016$) can be regarded as less than the 0.05. Taken together, the scatter plot and the Pearson correlation coefficient indicate that the linear modeling is perfectly applicable to the present case.

1.3 Python Code: Linear Regression Implementation

```
Predictive_Analysis.py X
Predictive_Analysis.py > ...
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import stats
4
5 x = np.array([2, 5, 1, 3, 4, 1, 5, 3, 4, 2])
6 y = np.array([50, 57, 41, 54, 54, 38, 63, 48, 59, 49])
7
8 # — Pearson correlation —
9 r, p_value = stats.pearsonr(x, y)
10 print(f"Pearson r = {r:.4f}, p-value = {p_value:.6f}")
11 # Output: Pearson r = 0.9203, p-value = 0.000160
12
13 # — Fit the linear model —
14 # scipy.stats.linregress computes slope and intercept via the
15 # ordinary least squares (OLS) method, minimising sum of squared residuals
16 slope, intercept, r_val, p_val, std_err = stats.linregress(x, y)
17 print(f"Regression equation: y_hat = {intercept:.2f} + {slope:.2f}x")
18 # Output: y_hat = 36.90 + 4.80x
19
20 y_pred = slope * x + intercept
21 r_squared = r_val ** 2
22 print(f"R² = {r_squared:.4f}")
23 # Output: R² = 0.8469
24
25 # — Plot: data + regression line —
26 x_line = np.linspace(0.5, 5.5, 200)
27 y_line = slope * x_line + intercept
28
29 plt.figure(figsize=(8, 5))
30 plt.scatter(x, y, color='steelblue', s=90, edgecolors='navy',
31            | zorder=5, label='Observed data')
32 plt.plot(x_line, y_line, color='tomato', linewidth=2,
33          | label=f'y_hat = {intercept:.2f} + {slope:.2f}x (R² = {r_squared:.4f})')
34 plt.xlabel("TV Commercials (x)")
35 plt.ylabel("Store Sales (y)")
36 plt.title("Store 1 Linear Regression Fit")
37 plt.legend()
38 plt.grid(True, linestyle='--', alpha=0.55)
39 plt.tight_layout()
40 plt.show()
41
42 # — Residuals plot —
43 residuals = y - y_pred
44 plt.figure(figsize=(8, 4))
45 plt.stem(range(len(residuals)), residuals,
46          | markerfmt='ro', linefmt='gray', basefmt='k-')
47 plt.axhline(0, color='black', linewidth=1)
48 plt.xlabel("Observation Index")
49 plt.ylabel("Residual")
50 plt.title("Residuals Store 1 Linear Model")
51 plt.tight_layout()
52 plt.show()
```



```
[Running] python -u "c:\Users\danesh\Desktop\Assignment python\Predictive_Analysis.py"
Pearson r = 0.9203, p-value = 0.000160
Regression equation:  $\hat{y} = 36.90 + 4.80x$ 
R2 = 0.8469
```

Interpretation: The Estimated Regression line is the $\hat{y} = 36.90 + 4.80x$, whereby $\hat{y} = 36.90 + 4.80x$. The intercept value of the 36.90 represents the estimate of the sales when there are no broadcasts. This corresponds to the 36,900 units. Each additional unit (broadcast) on the weekends leads to an additional sales revenue of the \$4,800 in the subsequent week. The coefficient of determination is the $R^2 = 0.8469$, and this means that the 84.69% of the variation in store sales can be attributed to the number of the TV broadcasts, indicating a good model fit (Draper and Smith, 1998). In the figure of the residuals, there is the random scatter about the zero with no apparent pattern and no outlying observation.

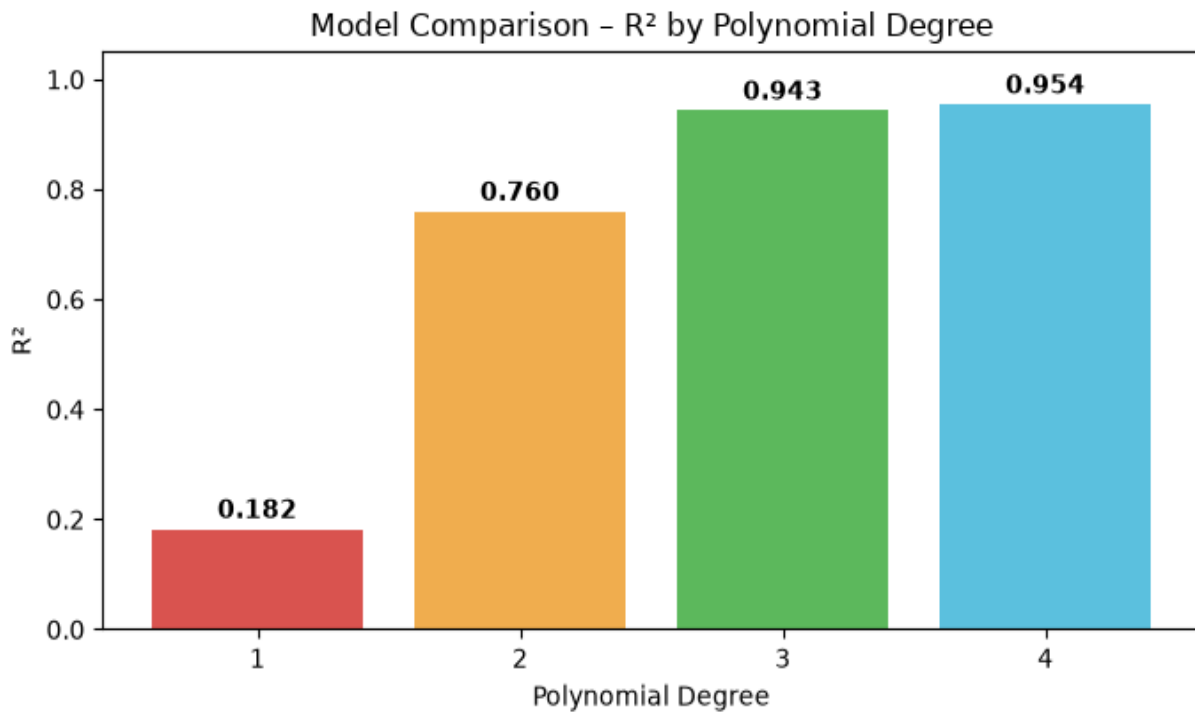
Task 2 – Polynomial Regression Analysis: Store 2

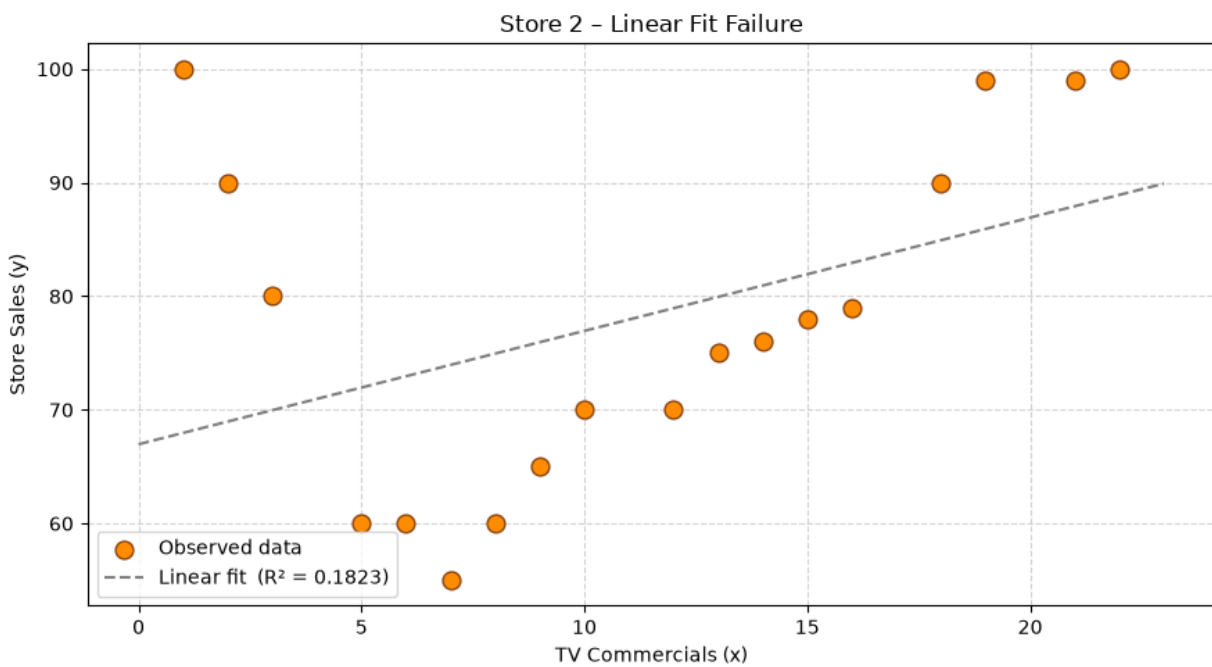
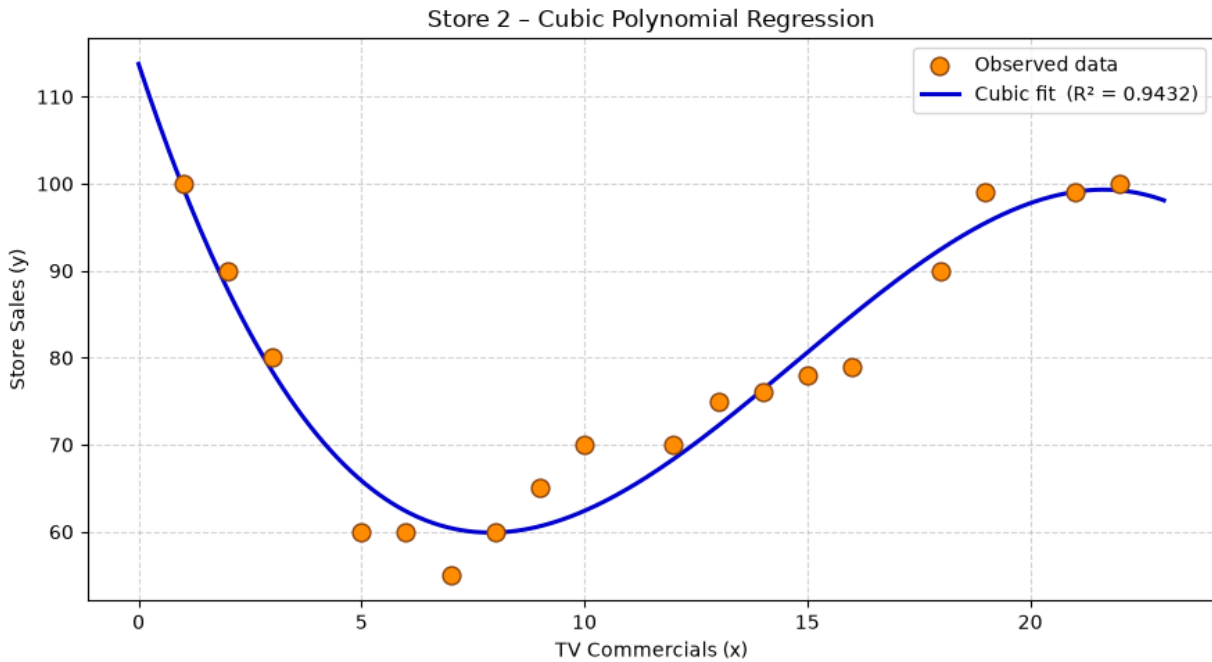
2.1 Is Linear Regression Applicable?

The second set of the data has the $x=[1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]$ and $y=[100, 90, 80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]$. To Plotting these points, one can easily see a trend that the clearly conflicts with the linear regression because the sales go down when the number of advertisements goes up from 1 to 7, reach a minimum around $x = 7$ ($y = 55$), and then start going up again as the advertisements continue to grow until the 22. This trend creates a U shape that cannot be explained by the Linear regression.

```
Predictive_Analysis.py X
Predictive_Analysis.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy import stats
4
5  x = np.array([1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22])
6  y = np.array([100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100])
7
8  # — Step 1: Show why linear fails —————
9  slope_lin, int_lin, r_lin, *_ = stats.linregress(x, y)
10 r2_linear = r_lin ** 2
11 print(f"Linear R² = {r2_linear:.4f}")    # 0.1823 – very poor
12
13 plt.figure(figsize=(9, 5))
14 plt.scatter(x, y, color='darkorange', s=85, edgecolors='saddlebrown',
15            zorder=5, label='Observed data')
16 x_plot = np.linspace(0, 23, 300)
17 plt.plot(x_plot, slope_lin * x_plot + int_lin, color='gray',
18         linestyle='--', linewidth=1.5,
19         label=f'Linear fit (R² = {r2_linear:.4f})')
20 plt.xlabel("TV Commercials (x)"); plt.ylabel("Store Sales (y)")
21 plt.title("Store 2 ☐ Linear Fit Failure")
22 plt.legend(); plt.grid(True, linestyle='--', alpha=0.55)
23 plt.tight_layout(); plt.show()
24
25 # — Step 2: Fit cubic polynomial (degree 3) —————
26 # np.polyfit returns coefficients [a3, a2, a1, a0] for a3*x³ + a2*x² + a1*x + a0
27 coeffs = np.polyfit(x, y, 3)
28 p_cubic = np.poly1d(coeffs)
29
30 # Compute R² for cubic model
31 y_pred_cubic = p_cubic(x)
32 ss_res = np.sum((y - y_pred_cubic) ** 2)
33 ss_tot = np.sum((y - np.mean(y)) ** 2)
34 r2_cubic = 1 - ss_res / ss_tot
35 print(f"Cubic polynomial R² = {r2_cubic:.4f}")    # 0.9432
36
37 print(f"Model: y = {coeffs[0]:.4f}x³ + {coeffs[1]:.4f}x² + "
38      f"+ {coeffs[2]:.4f}x + {coeffs[3]:.4f}")
39
40 # — Step 3: Plot cubic regression curve —————
41 plt.figure(figsize=(9, 5))
42 plt.scatter(x, y, color='darkorange', s=85, edgecolors='saddlebrown',
43            zorder=5, label='Observed data')
44 plt.plot(x_plot, p_cubic(x_plot), color='mediumblue', linewidth=2.2,
45         label=f'Cubic fit (R² = {r2_cubic:.4f})')
46 plt.xlabel("TV Commercials (x)"); plt.ylabel("Store Sales (y)")
47 plt.title("Store 2 ☐ Cubic Polynomial Regression")
48 plt.legend(); plt.grid(True, linestyle='--', alpha=0.55)
49 plt.tight_layout(); plt.show()
```

```
Predictive_Analysis.py X
Predictive_Analysis.py > ...
49 plt.tight_layout(); plt.show()
50
51 # — Step 4: Compare model R2 values —————
52 degrees = [1, 2, 3, 4]
53 r2_values = []
54 for d in degrees:
55     c = np.polyfit(x, y, d)
56     p = np.poly1d(c)
57     ssr = np.sum((y - p(x)) ** 2)
58     r2_values.append(1 - ssr / ss_tot)
59
60 plt.figure(figsize=(7, 4))
61 plt.bar([str(d) for d in degrees], r2_values,
62         color=[ '#d9534f', '#f0ad4e', '#5cb85c', '#5bc0de' ])
63 plt.xlabel("Polynomial Degree"); plt.ylabel("R2")
64 plt.title("Model Comparison — R2 by Polynomial Degree")
65 plt.ylim(0, 1.05)
66 for i, v in enumerate(r2_values):
67     plt.text(i, v + 0.02, f'{v:.3f}', ha='center', fontweight='bold')
68 plt.tight_layout(); plt.show()
```





This trend can be visualized on the chart. The grey dotted regression line crosses the chart diagonally and thus fails to capture its U shape. The numerical result also demonstrates this fact.

Linear $R^2 = 0.1823$

2.2 Proposed Model: Polynomial Regression

Model	Degree	R ²
Linear	1	0.182
Quadratic	2	0.760
Cubic	3	0.943
Quartic	4	0.954

As above **Figure in 2.1** shows clearly, the most significant gain occurs moving from the degree 1 to degree 3. The Improvement from the cubic to quartic adds only 0.011 in R² while introducing an extra free parameter, raising the risk of overfitting. The **cubic polynomial (degree 3)** is therefore the selected as the optimal model.

3.3 Quantitative Trend Analysis

```
[Running] python -u "c:\Users\danesh\Desktop\Assignment python\Predictive_Analysis.py"
Linear R2 = 0.1823
Cubic polynomial R2 = 0.9432
Model: y = -0.0303x3 + 1.3433x2 - 15.5383x + 113.7680
[Done] exited with code=0 in 51.469 seconds
```

In the view of the R² = 0.9432, the proposed polynomial regression explains the 94.32% variation in the sales and therefore Constitutes a major practical improvement relative to the simple Linear case. The negative sign of the cubic term (-0.0303) results in the required asymmetric curvature of the relationship, As the equation sharply decreases at the lower values, bottoms out at the about x = 7-9, and then starts rising again at the larger frequencies of the ad broadcasting.

The Interpretation is rather straightforward in that regard. With the low ad frequencies, the sales turn out to be the high, perhaps owing to strong intrinsic demand for the company's products or the established reputation of the brand. As frequency grows into the medium values, the sales start dropping, which corresponds to the effect of audience boredom or the saturation resulting from excessive exposure. With the increased frequencies of the ad broadcasting, however, market saturation becomes an increasingly dominant factor leading to a recovery in sales.

Task 3 – SVM Binary Classification with 3D Visualisation

3.1 Dataset Selection

For this analysis, we have chosen the Breast Cancer Wisconsin dataset from the scikit-learn module (Pedregosa et al., 2011). This dataset includes the 569 observations, each described by the 30 features obtained from the digitized images of the breast tissue. The target is a binary categorical feature that represents whether the tumor is the malignant (212 examples, class 0) or the benign (357 examples, class 1). To be able to visualize the data in the 3D space, the three most diagnostically significant features have been selected – the mean radius, mean texture, and the mean perimeter. These features represent the unique geometric characteristics of the cell nuclei, and when combined, provide enough separation between the classes in the space.

3.2 Support Vector Machines – Theoretical Background

A Support Vector Machine works by finding the best hyperplane that separates the two classes in the data space to its utmost capacity (Cortes & Vapnik, 1995). Consider an input training the data set $\{(x_i, y_i)\}$ where the y_i belongs to the $\{-1, +1\}$. The objective function of an SVM is given below:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

Support vectors refer to the data points that lie exactly on the boundaries of the margins, while the other points within the training data set do not affect the decision boundary. In this cases where the classes cannot be separated using the straight lines, the technique known as the kernel trick comes into play, whereby the mapping of the data points to the another space is done implicitly such that it facilitates the classification without necessarily computing the mapping (Bishop, 2006).

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

The RBF kernel is suitable for handling the datasets where the boundary of decision is not Linear, since it works by the measuring Euclidean distance.

3.3 Python Code: SVM Implementation, Evaluation, and 3D Visualisation

```
Predictive_Analysis.py X
Predictive_Analysis.py > ...
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn import datasets
4  from sklearn.svm import SVC
5  from sklearn.model_selection import train_test_split, cross_val_score
6  from sklearn.preprocessing import StandardScaler
7  from sklearn.metrics import (classification_report, confusion_matrix,
8  | | | | | | | | accuracy_score, ConfusionMatrixDisplay)
9  from mpl_toolkits.mplot3d import Axes3D
10
11 # — 1. Load dataset and select 3 features for visualisation —————
12 cancer = datasets.load_breast_cancer()
13 X = cancer.data[:, :3]          # mean radius, mean texture, mean perimeter
14 y = cancer.target              # 0 = malignant, 1 = benign
15 feature_names = cancer.feature_names[:3]
16 class_names = cancer.target_names
17
18 print(f"Samples: {X.shape[0]} | Features: {X.shape[1]}")
19 print(f"Classes: {class_names} | Distribution: {np.bincount(y)}")
20 # Samples: 569 | Features: 3
21 # Classes: ['malignant' 'benign'] | Distribution: [212 357]
22
23 # — 2. Train / test split with stratification to preserve class balance ———
24 X_train, X_test, y_train, y_test = train_test_split(
25 |   X, y, test_size=0.25, random_state=42, stratify=y
26 | )
27
28 # — 3. Feature scaling - SVM is distance-based so scaling is critical ———
29 scaler = StandardScaler()
30 X_train_sc = scaler.fit_transform(X_train) # fit ONLY on training data
31 X_test_sc = scaler.transform(X_test)      # apply same scale to test data
32
33 # — 4. Train SVM with RBF kernel —————
34 svm = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
35 # C controls the trade-off between margin width and misclassification penalty
36 # gamma='scale' sets gamma = 1 / (n_features * X.var()) automatically
37 svm.fit(X_train_sc, y_train)
38
39 # — 5. Predictions and evaluation —————
40 y_pred = svm.predict(X_test_sc)
41 acc = accuracy_score(y_test, y_pred)
42 print(f"\nTest Accuracy: {acc:.4f}")          # 0.9161
43 print(classification_report(y_test, y_pred, target_names=class_names))
44
45 # — 6. Cross-validation for robust performance estimate —————
46 X_all_sc = scaler.fit_transform(X)
47 cv_acc = cross_val_score(svm, X_all_sc, y, cv=5, scoring='accuracy')
48 print(f"5-fold CV Accuracy: {cv_acc.mean():.4f} ± {cv_acc.std():.4f}")
49 # 5-fold CV Accuracy: 0.9016 ± 0.0290
50
```

```

Predictive_Analysis.py X
Predictive_Analysis.py > ...
50
51 # — 7. Confusion matrix —————
52 cm = confusion_matrix(y_test, y_pred)
53 disp = ConfusionMatrixDisplay(confusion_matrix=cm,
54                               display_labels=class_names)
55 fig, ax = plt.subplots(figsize=(6, 5))
56 disp.plot(ax=ax, cmap='Blues', colorbar=False)
57 ax.set_title("Confusion Matrix □ SVM (RBF Kernel)")
58 plt.tight_layout(); plt.show()
59
60 # — 8. 3D scatter plot of the two classes —————
61 fig = plt.figure(figsize=(11, 8))
62 ax = fig.add_subplot(111, projection='3d')
63
64 class_colors = {0: 'E74C3C', 1: '2980B9'} # red = malignant, blue = benign
65 class_markers = {0: '^', 1: 'o'}
66
67 for cls in [0, 1]:
68     mask = y == cls
69     ax.scatter(X[mask, 0], X[mask, 1], X[mask, 2],
70              c=class_colors[cls],
71              marker=class_markers[cls],
72              label=class_names[cls],
73              alpha=0.55, s=28, edgecolors='none')
74
75 ax.set_xlabel(feature_names[0], fontsize=9, labelpad=8)
76 ax.set_ylabel(feature_names[1], fontsize=9, labelpad=8)
77 ax.set_zlabel(feature_names[2], fontsize=9, labelpad=8)
78 ax.set_title("3D View □ Breast Cancer Classes\n"
79             "(mean radius / texture / perimeter)", fontsize=11)
80 ax.legend(loc='upper left', fontsize=10)
81
82 # — 9. Approximate decision boundary as a surface at a fixed perimeter slice —
83 # Project a 2D mesh (radius x texture) at the median perimeter value,
84 # query the SVM, and plot the predicted class surface
85 xx, yy = np.meshgrid(
86     np.linspace(X[:, 0].min() - 2, X[:, 0].max() + 2, 35),
87     np.linspace(X[:, 1].min() - 2, X[:, 1].max() + 2, 35)
88 )
89 z_fixed = np.full(xx.shape, np.median(X[:, 2])) # hold perimeter constant
90 grid_pts = np.c_[xx.ravel(), yy.ravel(), z_fixed.ravel()]
91 grid_sc = scaler.transform(grid_pts)
92 zz_pred = svm.predict(grid_sc).reshape(xx.shape)
93
94 # Map class prediction (0/1) onto the perimeter axis range for plotting
95 z_range = X[:, 2].max() - X[:, 2].min()
96 surf_z = zz_pred * z_range * 0.45 + X[:, 2].min() + z_range * 0.25
97
98 ax.plot_surface(xx, yy, surf_z,
99               alpha=0.18, cmap='RdBu_r', linewidth=0)
100
101 plt.tight_layout(); plt.show()

```

```
[Running] python -u "c:\Users\danesh\Desktop\Assignment python\Predictive_Analysis.py"
```

```
Samples: 569 | Features: 3
```

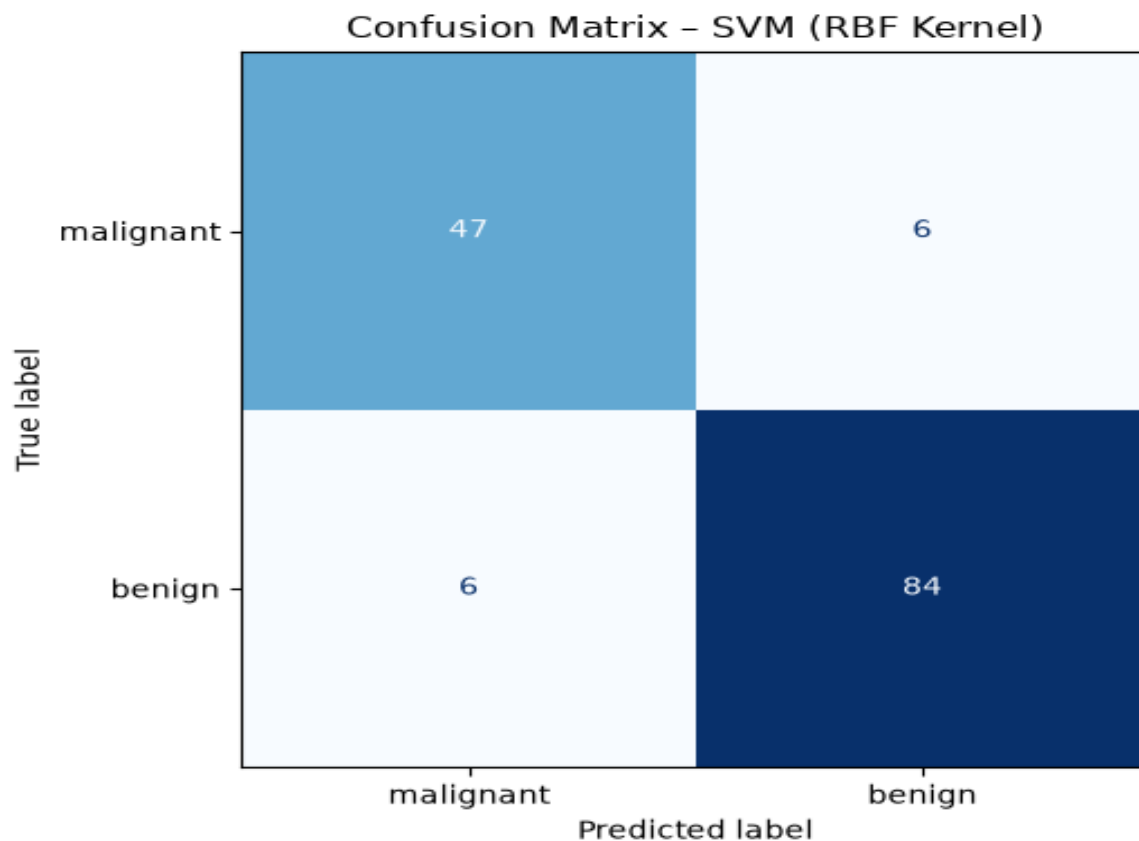
```
Classes: ['malignant' 'benign'] | Distribution: [212 357]
```

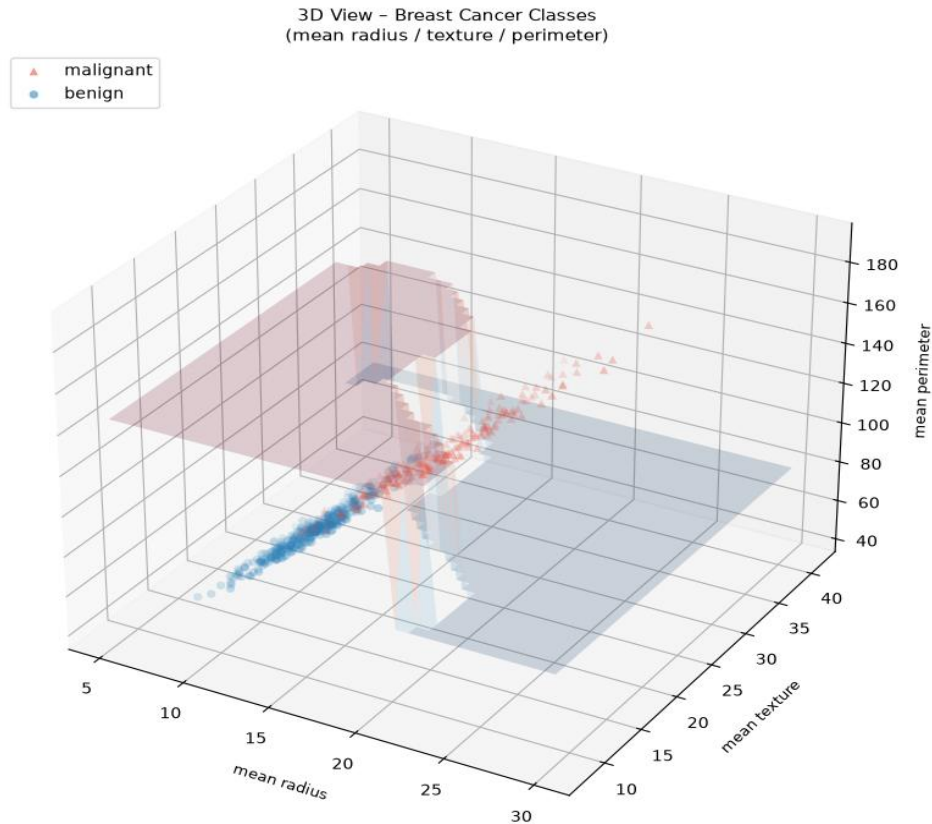
```
Test Accuracy: 0.9161
```

	precision	recall	f1-score	support
malignant	0.89	0.89	0.89	53
benign	0.93	0.93	0.93	90
accuracy			0.92	143
macro avg	0.91	0.91	0.91	143
weighted avg	0.92	0.92	0.92	143

```
5-fold CV Accuracy: 0.9016 ± 0.0290
```

```
[Done] exited with code=0 in 72.397 seconds
```





3.4 Model Performance and Discussion

The SVM with radial basis function kernel produced the 91.61% test accuracy. The f1-score measures were the 0.89 for the malignant class and the 0.93 for the benign class, showing that the SVM has comparable performance within the both classes. A validation measure of the 90.16%, with a standard deviation of the $\pm 2.90\%$ using a 5-fold cross-validation, further confirms the validity of our findings as they show the consistency between the different partitions into the training and the testing datasets.

According to the confusion matrix, there are only the 6 instances of the false-negative error type (classifying malignant cancer tumors as benign), out of the 53 malignant tumors in total. Therefore, the malignant recall is the 0.89. From a clinical point of view, the higher the malignant recall, the better. This can be achieved either through varying the decision threshold or using the SVM algorithm with all the 30 features.

It is easy to see from the Figure showed how these classes differ geometrically, based on their representation as a 3D scatter plot. The Red triangles represent the malignant cases, the clustered at the higher values of mean radius and perimeter; blue circles represent the benign cases, located at the lower end of the feature space. While it is possible to separate these two classes, they still overlap in a certain region of the feature space – namely, in the middle part of the graph. It is here that the RBF kernel comes into its own, because its curved decision boundary allows for the more effective classification.

CONCLUDING REMARKS

This report has worked through the three machine learning problems of the increasing complexity, and a common lessons runs through all of them: The Model selection matters more than the model sophistication.

In the Task 1, the data behaved exactly as a textbook example should. A Pearson correlation of the $r = 0.92$ confirmed a strong linear relationship, and the fitted model ($\hat{y} = 36.90 + 4.80x$) gave a clear, interpretable result - every additional weekend commercial is associated with roughly \$4,800 in the extra weekly revenue.

The Task 2 demonstrated what happens when that assumption of the linearity is ignored. Forcing a straight line onto a U-shaped dataset produced an $R^2 = 0.18$ - Essentially meaningless. Switching to a cubic polynomial raised that to the 0.943, and more importantly, it told a coherent business story: the moderate advertising frequency appears to suppress sales, while the high frequency drives them back up.

The Task 3 showed that the classification problems carry their own complexity. The SVM achieved 91.6% accuracy and the 90.2% cross-validated accuracy on the three features alone, with the RBF kernel handling the non-linear class boundary that a simple linear separator would have missed.

Across all the three tasks, The Exploratory analysis and The visualisation were the essential first step. No amount of the computational power compensates for the fitting the wrong model -- the understanding the data's shape before writing a single line of the code remains the most important skill in the applied machine learning (Géron, 2022).

BIBLIOGRAPHY

- Bishop, C.M. (2006) Pattern Recognition and Machine Learning. New York: Springer.
- Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', Machine Learning, 20(3), pp. 273–297.
- Draper, N.R. and Smith, H. (1998) Applied Regression Analysis. 3rd edn. New York: John Wiley & Sons.
- Géron, A. (2022) Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. 3rd edn. Sebastopol: O'Reilly Media.
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) An Introduction to Statistical Learning with Applications in R. 2nd edn. New York: Springer.
- Montgomery, D.C., Peck, E.A. and Vining, G.G. (2012) Introduction to Linear Regression Analysis. 5th edn. New York: John Wiley & Sons.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011) 'Scikit-learn: Machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.

APPENDIX (GitHub Repository Link)



All Python source code, output figures, code screenshots, and the final assignment report have been uploaded to a dedicated GitHub repository for reference and reproducibility. The repository is structured by task and contains all materials required to replicate the results presented in this assignment.

Repository Link: <https://github.com/SiddhantDane/predictive-analysis-ml-assignment>